TileLang + Batch-Invariant Kernels

Same token, same bits, in training and serving

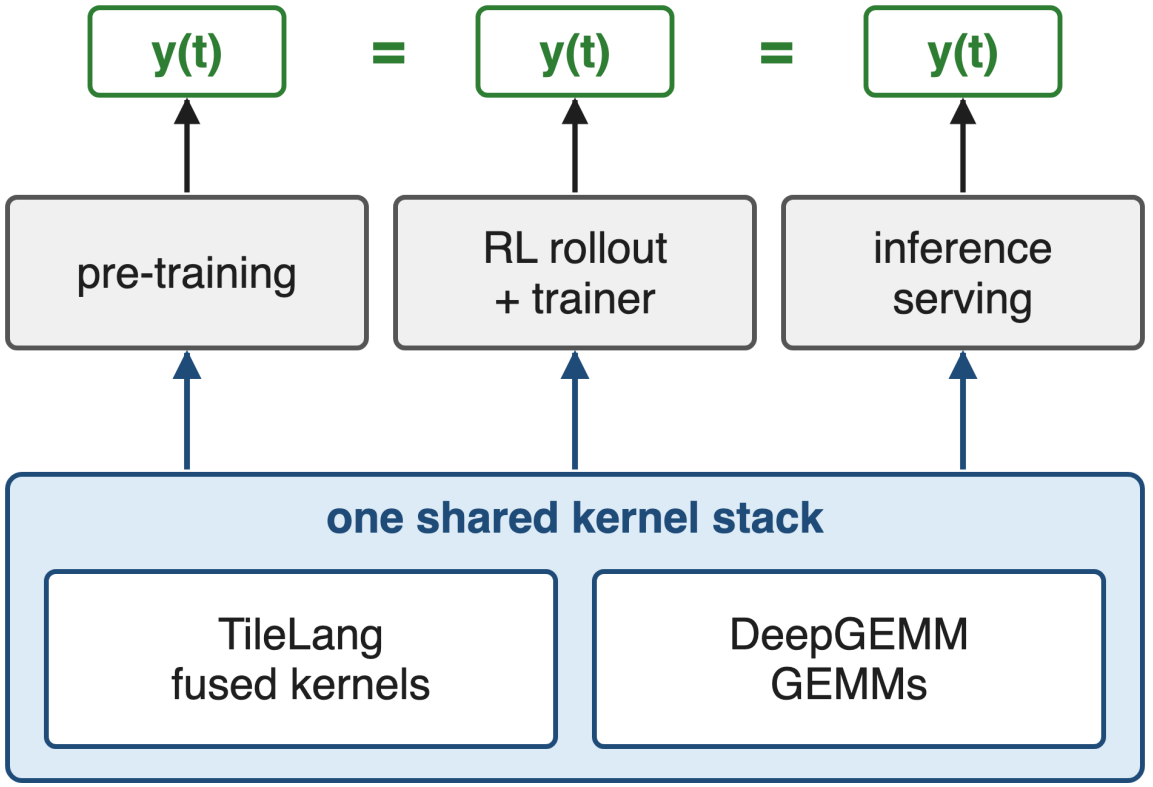
DeepSeek-AI · DeepSeek-V4

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Kernel Fusion & Runtime · arxiv.org/abs/2606.19348

Pre-training, RL and serving must agree bit for bit

- V4-Pro: **1.6T** params, **49B** active
- New layers (CSA/HCA, mHC) would mean hundreds of small ATen ops
- One kernel stack serves every pipeline



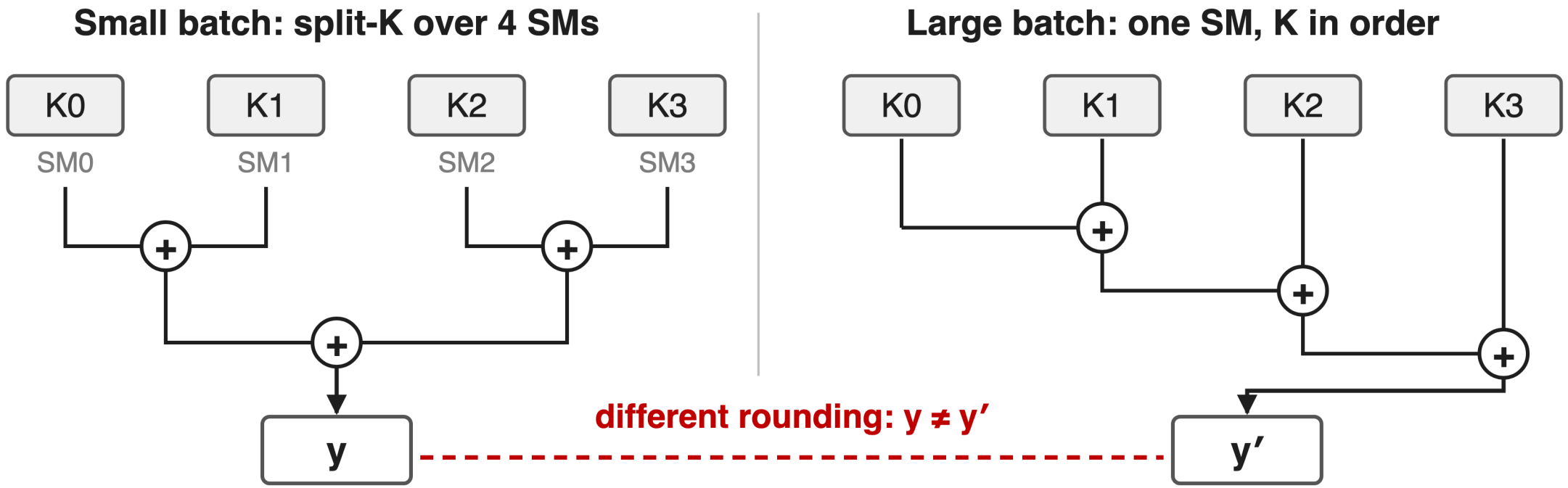
Same request, 80 different answers

80 / 1000

distinct completions from 1000 identical temperature-0 requests to Qwen3-235B served by vLLM

Thinking Machines Lab, "Defeating Nondeterminism in LLM Inference" (Sep 2025)

Reduction order follows the batch, not the token

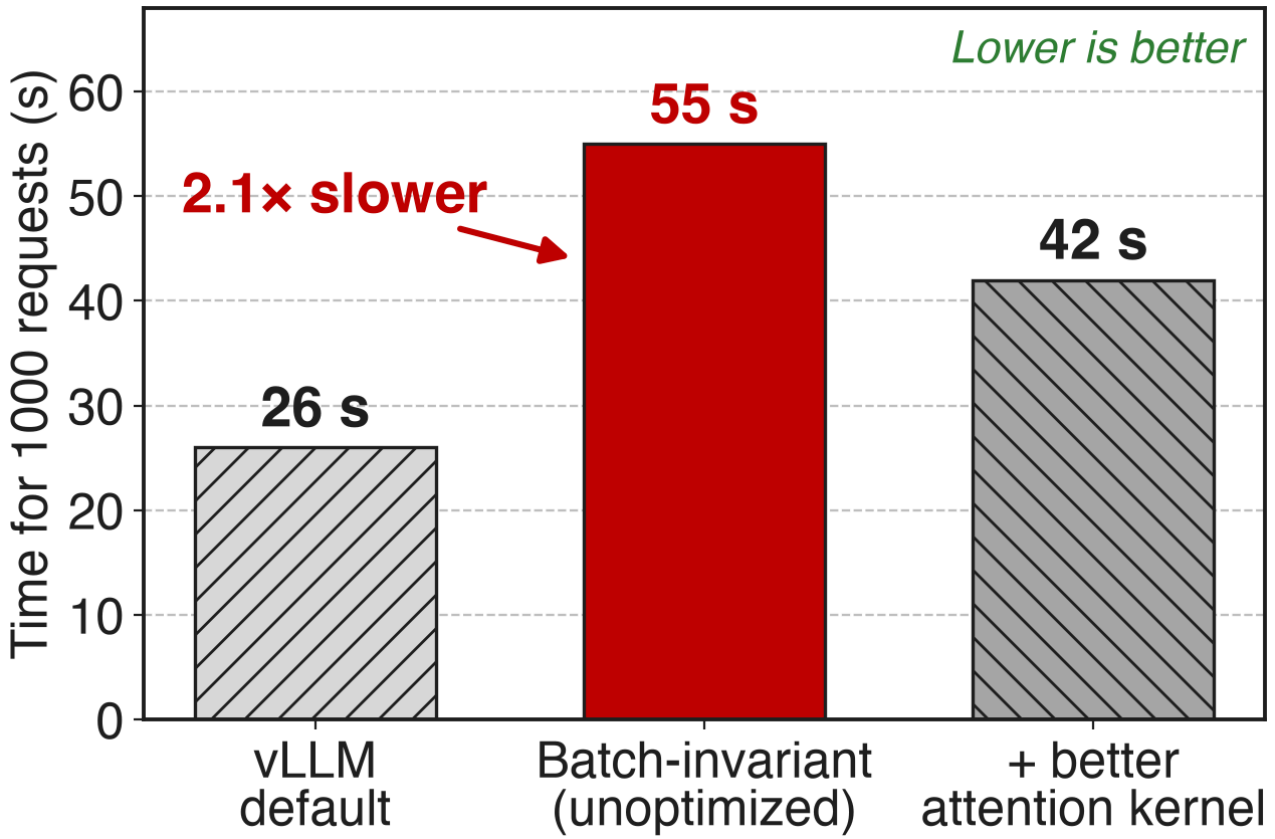


K0–K3: partial dot products over four chunks of the reduction dimension (schematic)

Float addition is not associative: batch-dependent splits and atomics change the bits

Naive batch invariance costs 2.1× serving time

- One fixed kernel config: no split-K, no split-KV
- Small batches and tail waves lose parallelism
- Better attention kernel: still **1.6×**



Thinking Machines, Qwen3-8B on one GPU; ratios derived

Insight: fix the order, then add parallelism back

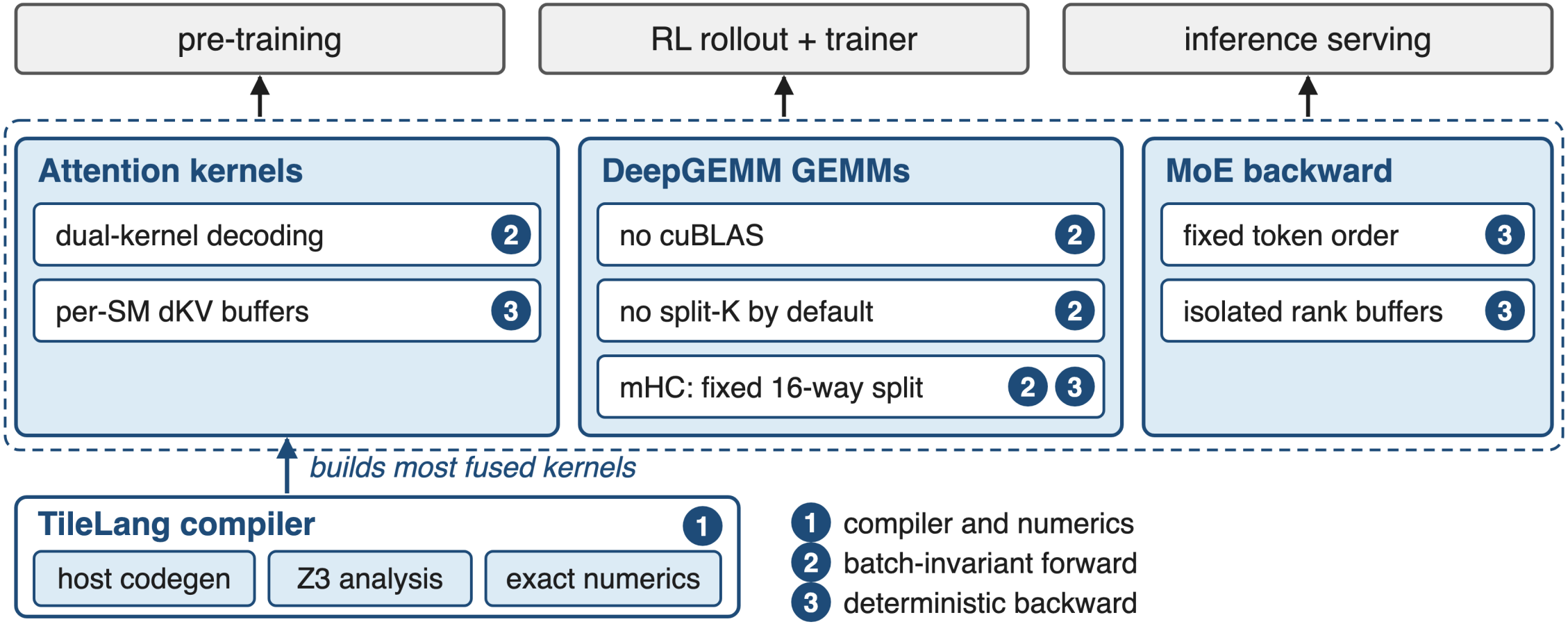
- A token's bits depend only on its accumulation order
- So fix that order per token, at every batch size

Insight: fix the order, then add parallelism back

- A token's bits depend only on its accumulation order
- So fix that order per token, at every batch size

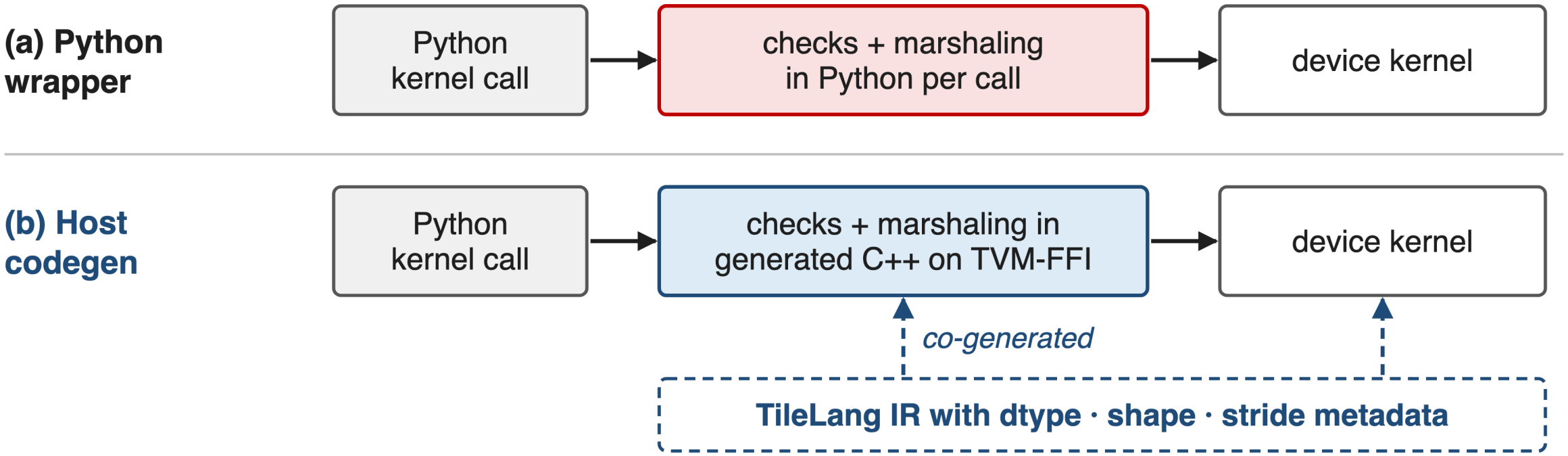
Regain parallelism without reordering: spread tails across SMs, sum per-SM buffers in order

Three layers make V4's kernels reproducible



Where each fix lives in DeepSeek-V4 (§3.2–3.3)

❶ TileLang generates the host launcher too



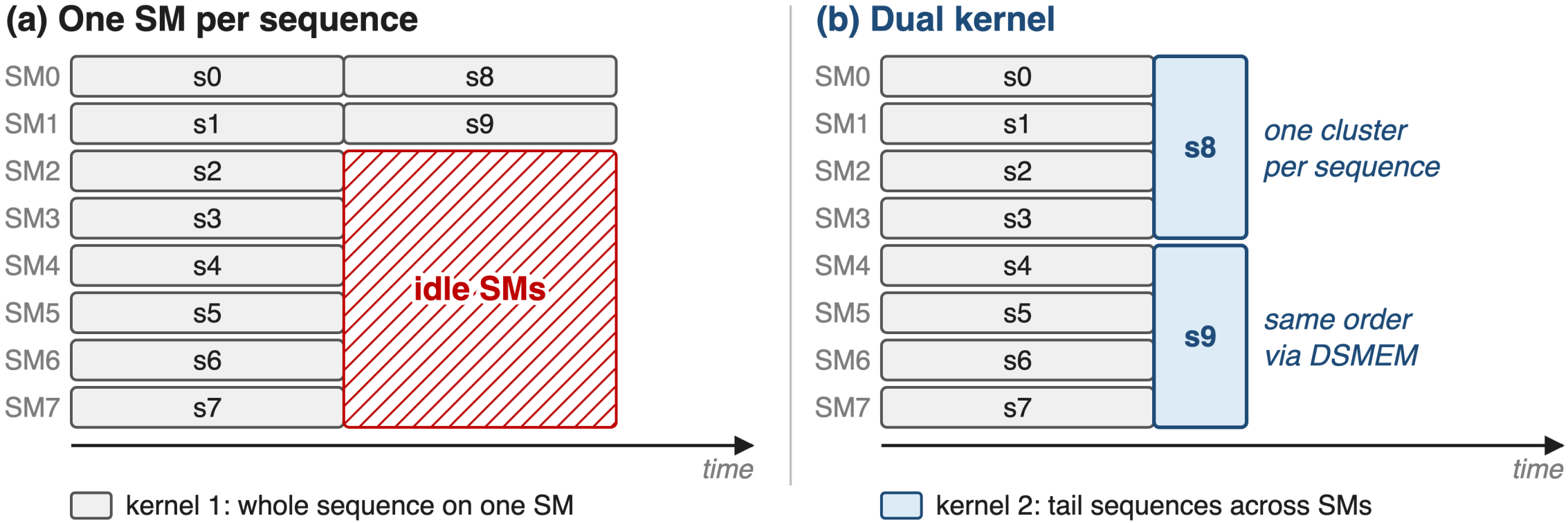
Per-call checks drop from tens–hundreds of µs in Python to under 1 µs

❶ TileLang keeps numerics exact by default

- **Exact by default**: fast-math off
- Approximations opt-in: `T.__exp`, `T.__log`
- IEEE intrinsics: `T.ieee_fdiv`, explicit rounding
- **Bit-identical to CUDA**: NVCC-aligned lowering
- `T.annotate_layout` pins accumulation order
- Z3 checks index math in seconds

A TileLang kernel can be tested bit for bit against its CUDA reference

❷ Decode attention splits only the tail wave



No split-KV: the tail kernel follows kernel 1's accumulation order exactly

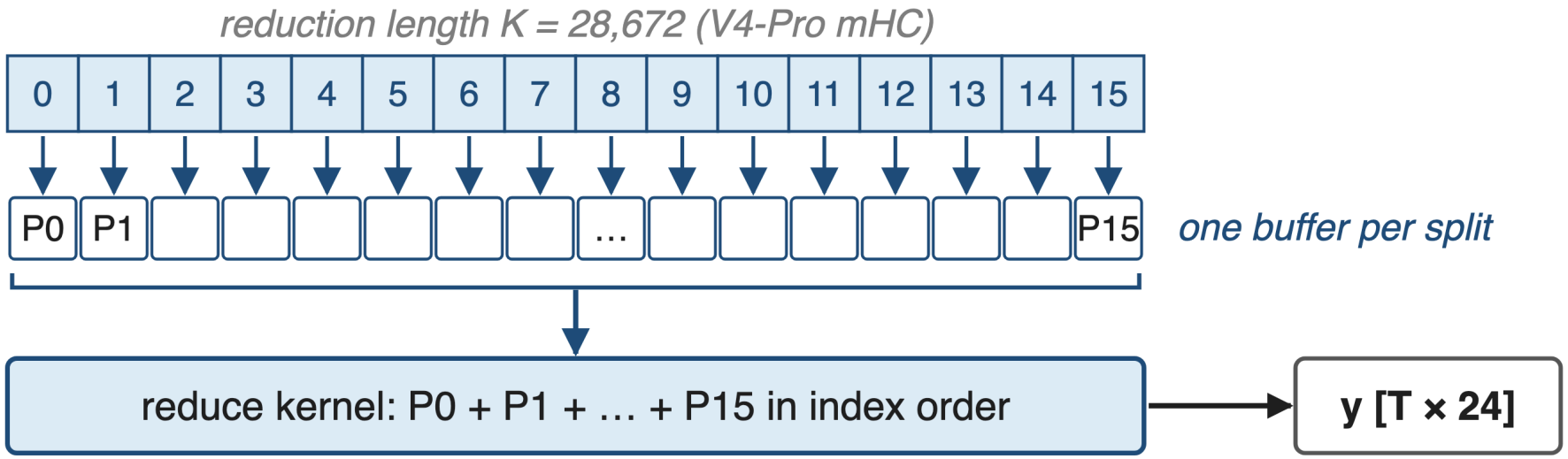
❷ GEMMs keep one K order at every batch

(a) Default
per-call splits



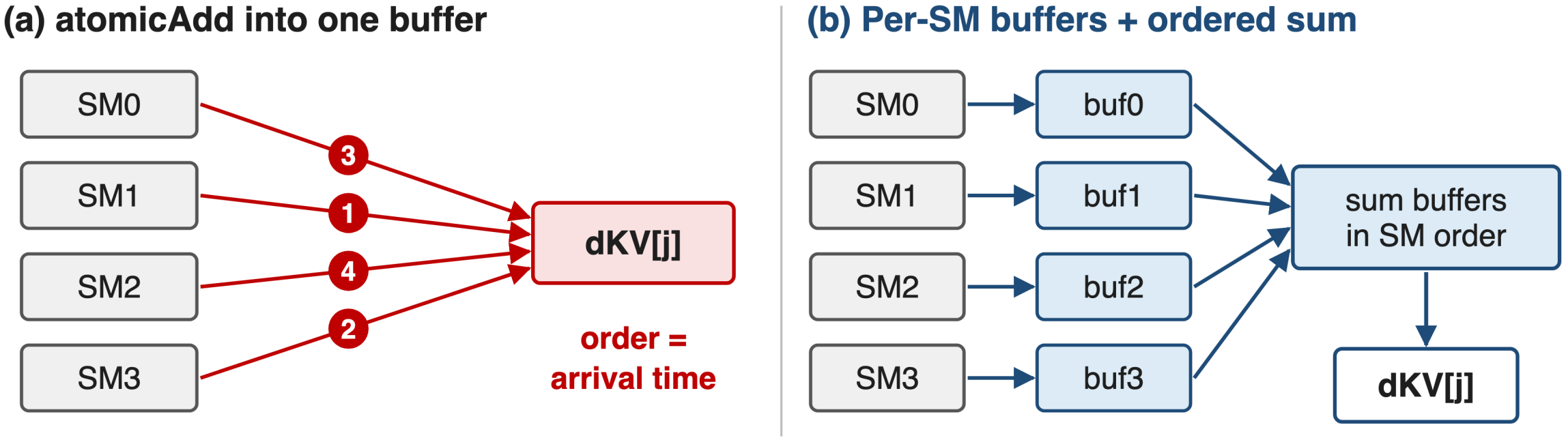
split count depends on batch T

(b) Deterministic
fixed 16 splits



Elsewhere: DeepGEMM replaces cuBLAS end to end and drops split-K

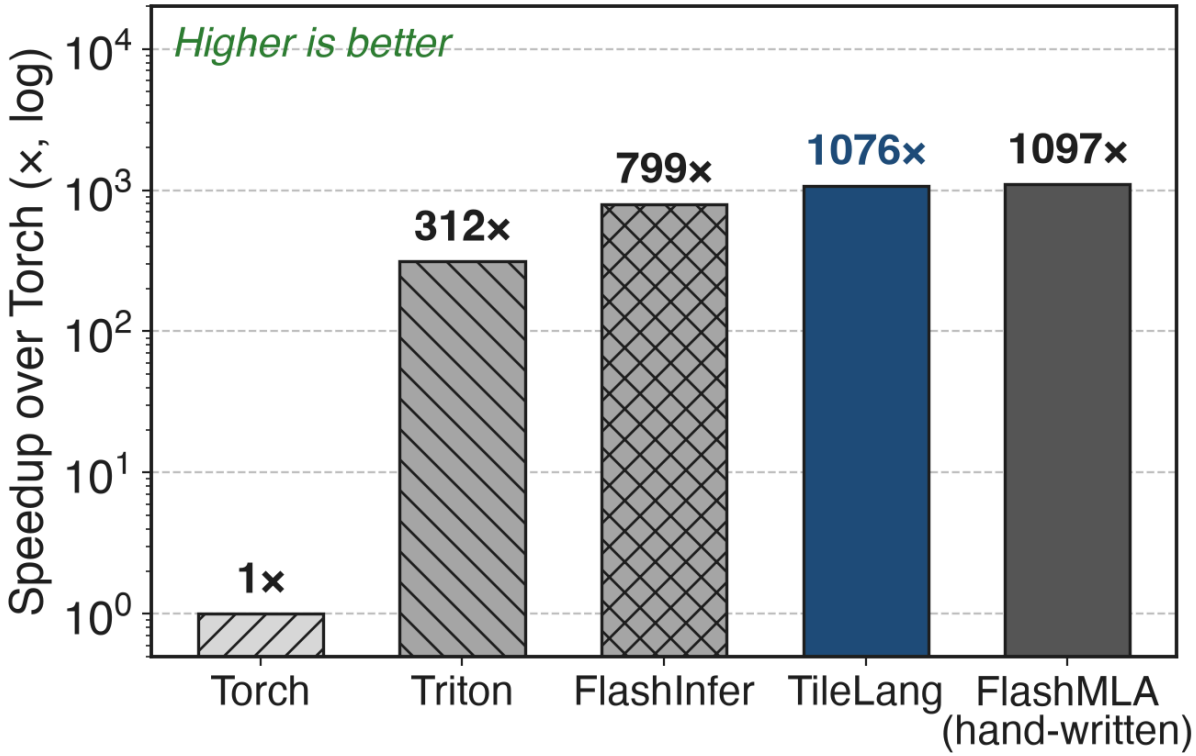
❸ Backward sums in a fixed order, not atomically



MoE backward: a fixed token order and per-rank buffers pin every write position

TileLang reaches 98% of FlashMLA in ~70 lines

- MLA decoding on H100 (TileLang paper)
- Hand-written FlashMLA: **1097×** over Torch
- TileLang: **1076×**, about **70** lines of Python



The DSL costs little speed: 98% of the hand-written kernel

The cost: own kernels, lost split-K, no numbers

- **No published overhead**: only "minimal"
- **Split-K dropped** in most GEMMs
- Every GEMM from DeepGEMM, none from cuBLAS
- Tail kernel needs clusters and DSMEM
- Invariant to batch position, not parallel layout
- Z3 adds seconds of compile time

Summary

- Non-invariant kernels: **80 / 1000** distinct answers (Qwen3, vLLM)
- **Insight**: fix each token's accumulation order, add parallelism around it
- TileLang: host checks **< 1 µs**, **98%** of FlashMLA
- V4: batch-invariant forward, deterministic backward

https://arxiv.org/abs/2606.19348